Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 3

з дисципліни „ Професійний Python”

на тему „ Ітератори, генератори та потокова обробка даних.”

Виконала:
студентка 3 курсу
групи ФеП-32
Клімащук Вероніка Ігорівна
Перевірила:
Доц. Куньо Іван Михайлович

Львів – 2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

2. Індивідуальний варіант

- **Варіант №9:** «Потокова обробка спортивних результатів».
- **Вхідні дані (CSV):** athlete, sport, result, category, timestamp.
- **Основні завдання:**
  - Streaming CSV reading та validation спортивних результатів.
  - Фільтрація за видом спорту та категорією.
  - Визначення найкращого результату.
  - Streaming average result.
  - Формування рейтингу результатів.
  - Pairwise аналіз зміни послідовних результатів (itertools.pairwise).
  - Batch processing записів.
  - Обмеження потоків через itertools.islice.

3. Структура проєкту (src-layout)

```
sports_streaming/
├── pyproject.toml
├── README.md
├── data/
│   └── sports_100k.csv
└── src/
    └── sports_streaming/
        ├── __init__.py
        ├── readers.py
        ├── parsers.py
        ├── models.py
        ├── filters.py
        ├── transformations.py
        ├── analytics.py
        ├── batches.py
        ├── pipeline.py
        └── main.py
```

4. Основні модулі програми

Models.py

sports_streaming > src > sports_streaming > models.py

```python
from typing import NamedTuple

class SportRecord(NamedTuple):
    athlete: str
    sport: str
    result: float
    category: str
    timestamp: str
```

Readers.py

```python
from pathlib import Path
from collections.abc import Iterator

def read_lines(path: Path) -> Iterator[str]:
    """Стрімінгове читання рядків файлу (використання yield from)."""
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

Parsers.py

```python
import csv
from collections.abc import Iterable, Iterator

def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    """Парсинг рядків CSV у словники через csv.DictReader (lazy)."""
    reader = csv.DictReader(lines)
    yield from reader
```

filters.py

sports_streaming > src > sports_streaming > filters.py > filter_by_sport_and_category

```python
from collections.abc import Iterable, Iterator
from sports_streaming.models import SportRecord

def validate_sports_records(rows: Iterable[dict[str, str]]) -> Iterator[SportRecord]:
    """Потокова валідація та очищення записів спортивних результатів."""
    for row in rows:
        try:
            athlete = row["athlete"].strip()
            sport = row["sport"].strip()
            result = float(row["result"])
            category = row["category"].strip()
            timestamp = row["timestamp"].strip()
        except (ValueError, KeyError, TypeError):
            continue

        if not athlete or not sport or not category or not timestamp:
            continue
        if result < 0:
            continue

        yield SportRecord(
            athlete=athlete,
            sport=sport,
            result=result,
            category=category,
            timestamp=timestamp
        )

def filter_by_sport_and_category(
    records: Iterable[SportRecord],
    sport: str | None = None,
    category_filter: str | None = None
) -> Iterator[SportRecord]:
    """Фільтрація записів за видом спорту та категорією."""
    for record in records:
        if sport and record.sport.lower() != sport.lower():
            continue
        if category_filter and record.category.lower() != category_filter.lower():
            continue
        yield record
```

analytics.py

```python
from collections.abc import Iterable
from sports_streaming.models import SportRecord

def streaming_average_result(records: Iterable[SportRecord]) -> float:
    """Потокове обчислення середнього результату без матеріалізації."""
    total = 0.0
    count = 0
    for record in records:
        total += record.result
        count += 1
    return total / count if count > 0 else 0.0

def find_best_result(records: Iterable[SportRecord]) -> SportRecord | None:
    """Визначення найкращого результату у потоці."""
    best = None
    for record in records:
        if best is None or record.result > best.result:
            best = record
    return best

def streaming_min_max(records: Iterable[SportRecord]) -> tuple[float | None, float | None]:
    """Визначення мінімального та максимального результатів."""
    minimum = None
    maximum = None
    for record in records:
        val = record.result
        if minimum is None or val < minimum:
            minimum = val
        if maximum is None or val > maximum:
            maximum = val
    return minimum, maximum
```

5. Результати експериментальної частини (Benchmark)

Порівняння Eager (повна матеріалізація у пам'ять) та Lazy (потокова обробка через генератори) підходів за допомогою tracemalloc та time.perf_counter():

Кількість записів	Eager Час (с)	Lazy Час (с)	Eager Пік пам'яті (MB)	Lazy Пік пам'яті (MB)
10 000	0.0185	0.0121	2.45 MB	0.08 MB
100 000	0.1742	0.1105	24.10 MB	0.09 MB
500 000	0.8920	0.5432	118.50 MB	0.10 MB

6. Висновок

У ході виконання лабораторної роботи №3 було успішно освоєно ітераційну модель Python, протокол ітераторів, використання генераторів із ключовим словом yield та yield from, а також ефективні інструменти модуля itertools. Побудований потоковий конвеєр (lazy pipeline) дозволив обробляти набори даних обсягом понад 500 000 записів із мінімальним сталим споживанням оперативної пам'яті (~0.1 MB), що підтверджує переваги принципів lazy evaluation порівняно з егерною матеріалізацією.